UT 2 · Acceder y manipular datos con Python

Apuntes para el aula · 2.º DAW · Python 3.11+ · VS Code · Git/GitHub · GitHub Copilot Pro

---

0. Cómo usar estos apuntes

En desarrollo web el backend casi nunca trabaja solo: lee ficheros de configuración, procesa CSV o Excel que suben los usuarios, recibe y devuelve JSON en APIs REST y transforma información antes de guardarla en una base de datos. Esta unidad te enseña a **guardar, recuperar, limpiar, transformar y analizar datos** con Python, empezando por un script de 5 líneas y terminando con un mini proceso ETL profesional.

Filosofía de la unidad: primero *comprender* los datos y las operaciones; después *automatizar*; por último *organizar un proceso profesional*. Copilot te ayuda a ir más rápido, pero **no sustituye** que entiendas lo que entregas.

Niveles de dificultad

Icono	Nivel	Qué se espera
🟢	Básico	Seguir el ejemplo guiado y adaptarlo
🟡	Consolidación	Resolver con autonomía combinando varios conceptos
🔴	Reto / ampliación	Problema abierto, buenas prácticas de empresa

Estructura de cada bloque

1. **¿Qué problema resuelve?** (para qué sirve en un proyecto real)
2. **Teoría breve**
3. **Ejemplo guiado** (pequeño y ejecutable)
4. 🤖 **Copilot** (cómo usarlo y qué auditar en ese bloque)
5. **Prácticas** 🟢 🟡 🔴
6. **Errores típicos**

Mapa de la unidad

Bloque	Contenido	Nivel	Sesiones orientativas
0	Entorno profesional, Git y Copilot	🟢	2
1	Archivos y persistencia	🟢	1
2	Lectura y escritura de texto (.txt, .log)	🟢	2
3	Rutas y directorios con `pathlib`	🟢🟡	2
4	CSV: `csv`, funciones, `try/except`	🟡	3
5	JSON y APIs con `requests`	🟡	3
6	pandas y DataFrames	🟡	3
7	Excel con pandas y openpyxl	🟡	2
8	Limpieza y transformación (“datos sucios”)	🟡🔴	3
9	Filtrado, agrupación y agregación	🔴	3
10	Proyecto integrador: mini ETL	🔴	4

Hilo conductor: el inventario informático

Todos los bloques usan el mismo caso: **el parque informático de una empresa**. Así reutilizas datos y conceptos de un bloque a otro. Usaremos un único esquema de columnas en todos los ficheros:

`codigo, dispositivo, marca, modelo, ram, precio, departamento, estado`

Crea `datos/equipos.csv` con este contenido (lo usaremos desde el Bloque 4):

```
codigo,dispositivo,marca,modelo,ram,precio,departamento,estado
PC01,Sobremesa,Lenovo,ThinkCentre M70,16,650,Administración,Disponible
PC02,Sobremesa,HP,ProDesk 400,32,850,Sistemas,En uso
PC03,Sobremesa,Dell,OptiPlex 7010,16,720,Administración,Disponible
PC04,Portátil,Lenovo,ThinkPad E14,16,780,Ventas,En uso
PC05,Portátil,HP,ProBook 450,32,990,Sistemas,En uso
PC06,Portátil,Dell,Latitude 5440,16,870,Ventas,En reparación
PC07,Sobremesa,Lenovo,ThinkCentre M90,32,910,Sistemas,Disponible
PC08,Portátil,Asus,ExpertBook B1,8,540,Administración,En uso
PC09,Sobremesa,HP,EliteDesk 800,32,1100,Dirección,En uso
PC10,Portátil,Lenovo,ThinkPad T14,32,1250,Dirección,Disponible
```

---

1. Bloque 0 · Preparación del entorno profesional 🟢

¿Qué problema resuelve?

Antes de abrir un solo archivo necesitamos un entorno **aislado, reproducible y versionado**. El clásico "en mi ordenador funciona" casi siempre viene de librerías con versiones distintas o de un proyecto que nadie sabe cómo instalar.

1.1 Tecnologías de la unidad

Tecnología	Tipo	Para qué la usamos
Python 3.11+ (recomendado 3.12)	Lenguaje	Scripts, backends y análisis de datos
Visual Studio Code + extensión Python	Editor/IDE	Escribir, ejecutar y depurar
Git y GitHub	Control de versiones	Historial, trabajo en equipo y entrega
GitHub Copilot Pro	Asistente IA	Explicar, depurar, revisar y generar tests
`pathlib`, `csv`, `json`, `logging`	Biblioteca estándar	Ya vienen con Python (no se instalan)
`requests`	Externa	Consumir APIs REST
`pandas`, `openpyxl`	Externas	Análisis tabular y ficheros Excel `.xlsx`
`pytest`	Externa	Pruebas automáticas (proyecto final)
`uv`, Docker, GitHub Actions	Contexto profesional	Se presentan; no son objetivo inicial

1.2 ¿Por qué un entorno virtual (`.venv`) si `csv` y `json` ya vienen con Python?

Porque desde el Bloque 5 instalaremos librerías externas y porque **así se trabaja en una empresa**. Instalar todo en el Python global es como tener una única caja de herramientas para todos los proyectos.

Característica	Python global	Entorno virtual `.venv`
Dónde se instalan las librerías	En todo el ordenador	Dentro de la carpeta del proyecto
Versiones distintas a la vez	Imposible	Sí, una por proyecto
`requirements.txt`	"Sucio" (incluye todo lo del PC)	Limpio (solo lo del proyecto)
Si algo se rompe	Reparar o reinstalar Python	Borras `.venv` y la recreas en segundos

1.3 Puesta en marcha paso a paso

Abre la terminal integrada de VS Code (`Ctrl + ñ`) en la carpeta del proyecto:

```
# 1. Crear el entorno virtual
python -m venv .venv

# 2. Activarlo (Windows PowerShell)
.venv\Scripts\Activate.ps1
# Linux / macOS:  source .venv/bin/activate

# 3. Instalar las librerías de la unidad
python -m pip install pandas openpyxl requests

# 4. Guardar las dependencias
python -m pip freeze > requirements.txt

# 5. Reconstruir el entorno en otro equipo tras clonar el repositorio
python -m pip install -r requirements.txt
```

> **Si PowerShell bloquea la activación** ("la ejecución de scripts está deshabilitada"), ejecuta una vez: `Set-ExecutionPolicy -Scope CurrentUser RemoteSigned`.

Seleccionar el intérprete en VS Code: `Ctrl + Shift + P` → *Python: Select Interpreter* → elige el que indica `('.venv': venv)`. Si la terminal muestra `(.venv)` al inicio de la línea, estás dentro del entorno.

1.4 Estructura del proyecto

```
ut2_python_datos/
├── .venv/                  # Entorno virtual (NUNCA a GitHub)
├── .github/
|   └── copilot-instructions.md   # Reglas de Copilot para este repo
├── datos/                  # Entradas: alumnos.txt, equipos.csv,
inventario.json...
├── ejercicios/             # ejercicio01_texto.py, ejercicio02_rutas.py ...
├── resultados/             # Salidas generadas (CSV, JSON, Excel, logs)
├── .env                    # Claves privadas (NUNCA a GitHub)
├── .gitignore
├── requirements.txt
└── README.md
```

Contenido mínimo de `.gitignore`:

```
.venv/
venv/
__pycache__/
*.pyc
.env
resultados/
```

1.5 Git: el flujo mínimo que vas a repetir en toda la unidad

```
git init
git add .
git commit -m "Estructura inicial del proyecto"
git branch -M main
git remote add origin https://github.com/TU_USUARIO/ut2_python_datos.git
git push -u origin main
```

Regla de la unidad: un commit por cada práctica terminada, con un mensaje que explique *qué* cambia (`Añade lectura de equipos CSV`, no `cambios`). Nunca subas `.venv/`, `.env` ni datos personales reales.

1.6 ¿Cómo se trabaja hoy en una empresa? (contexto, no examen)

- **`uv`** (gestor escrito en Rust, mucho más rápido que `pip`) se está convirtiendo en estándar. Crea y gestiona el `.venv` por ti, guarda las dependencias en `pyproject.toml` y fija versiones exactas en `uv.lock`:

```
uv init
uv add pandas openpyxl requests
uv run main.py        # ejecuta dentro del entorno
uv sync               # reconstruye el entorno desde el lock
```

- **Docker / DevContainers:** `.venv` aísla las librerías de Python, pero no el sistema operativo. Un contenedor empaqueta Linux + Python + dependencias + código, y evita diferencias entre tu Windows y el servidor.
- **CI/CD (GitHub Actions):** al hacer `git push`, un servidor crea un entorno limpio, instala dependencias y pasa los tests (`pytest`) automáticamente antes de aceptar el cambio.

En esta unidad usamos `venv` + `pip` porque te permiten **ver** qué ocurre; la lógica de `uv` es idéntica.

1.7 🤖 GitHub Copilot Pro en VS Code: uso responsable

Copilot es un **compañero que sugiere**, no un compañero que hace el trabajo por ti. En esta unidad se usa de cuatro formas:

Herramienta	Qué hace	Cuándo usarla
Sugerencias *inline* (Tab)	Autocompleta mientras escribes	Código repetitivo que ya entiendes
Chat modo **Ask**	Responde dudas sin tocar archivos	Entender un error o una instrucción
Chat modo **Edit / Agent**	Modifica archivos	Solo en ampliaciones y siempre revisando el *diff*
Comandos `/explain`, `/fix`, `/tests`, `/doc`	Explicar, corregir, generar tests, documentar	Tras escribir tu propia versión

Puedes dar contexto con `#file:` o seleccionando código antes de preguntar.

Prompts recomendados

Objetivo	Prompt
Comprender	*Explícame esta instrucción con un ejemplo pequeño; no escribas todo el programa.*
Depurar	*Indica el error, su causa y la corrección mínima.*
Comprobar	*Revisa si mi código cumple este enunciado y qué casos faltan por probar.*

Objetivo	Prompt
Mejorar	*Propón una mejora de legibilidad sin introducir conceptos que no hemos visto.*
Probar	*/tests Genera casos de prueba, incluyendo datos incorrectos.*

Fichero `.github/copilot-instructions.md` (Copilot lo lee automáticamente en el repositorio; así "configuramos" la IA para el aula):

```
- Responde siempre en español.
- Usa Python 3.11+, pathlib (no os.path) y open(..., encoding='utf-8').
- Si pido ayuda, explica primero; no escribas el programa completo.
- No uses conceptos que no se hayan visto en clase salvo que lo indique.
- Nunca incluyas claves, tokens ni datos personales en el código.
```

Normas de uso (se evalúan):

1. No aceptes código que no puedas explicar línea a línea.
2. Ejecuta y prueba siempre lo que sugiere; Copilot también se equivoca.
3. No compartas claves, tokens ni datos personales en el chat.
4. Cada entrega incluye una **bitácora breve de uso de Copilot**:

Fecha	Qué le pedí	Qué sugirió	Qué comprobé o cambié
12/11	Explicar `newline=''`	Evita líneas en blanco duplicadas en CSV	Lo probé quitándolo y vi el error

Práctica 0 · Entorno y primer commit

- 🟢 Crea el proyecto con la estructura de 1.4, el `.venv`, el `.gitignore` y el `requirements.txt`. Selecciona el intérprete en VS Code.
- 🟢 Crea `ejercicios/ejercicio00_hola.py` que imprima el nombre del intérprete con `import sys; print(sys.executable)` y comprueba que apunta a `.venv`.
- 🟢 Haz el primer commit y súbelo a GitHub.
- 🟡 Crea `.github/copilot-instructions.md` con tus propias reglas y pide a Copilot (Ask) que te explique qué es `pip freeze`.
- 🔴 Instala `uv` y reproduce el entorno en una carpeta nueva con `uv init` y `uv add pandas`. Explica por escrito en qué se diferencia de `venv + pip`.

Errores típicos: olvidar activar el `.venv`; instalar con `pip` sin `python -m` (se instala en otro Python); subir `.venv/` a GitHub; abrir VS Code en una carpeta superior y no en la raíz del proyecto (las rutas relativas fallan).

---

2. Bloque 1 · Archivos y persistencia 🟢

¿Qué problema resuelve?

Las variables viven en la **memoria RAM**, que es volátil: al terminar el programa (o reiniciar el servidor) desaparecen. Para que los datos sobrevivan necesitamos **persistencia**: guardarlos en un soporte permanente.

```
nombre = 'Carlos'
edad = 20
print(nombre, edad)   # al cerrar el programa, nombre y edad dejan de existir
```

Teoría

Un **archivo** es un conjunto de datos en un dispositivo no volátil (SSD, disco, nube). La **persistencia** es conservar esos datos para usarlos después de cerrar el programa.

Almacenamiento	Ejemplo	¿Persiste?	Uso habitual en desarrollo web
Variable (RAM)	`nombre = 'Carlos'`	No	Cálculos durante una petición
Texto / LOG	`usuarios.txt`, `app.log`	Sí	Trazas de actividad y errores
CSV	`equipos.csv`	Sí	Importar/exportar tablas
JSON	`usuarios.json`	Sí	Intercambio frontend ↔ backend, APIs REST
Excel	`inventario.xlsx`	Sí	Informes para dirección
Base de datos	SQLite, PostgreSQL	Sí	Datos de la aplicación (concurrencia)

> **Idea fundamental:** los archivos permiten que los datos sobrevivan al programa. En la UT de bases de datos veremos cuándo un archivo deja de ser suficiente.

Práctica 1 (de análisis, sin código)

- 🟢 Un programa de gestión de equipos debe recordar: nombre del equipo, IP, usuario asignado y estado. Decide para cada dato **dónde** guardarlo (variable, TXT, CSV, JSON, BD) y justifica por qué.
- 🟡 Razona cuál de los formatos de la tabla usarías para: (a) un registro de errores del servidor, (b) la respuesta de una API a tu frontend, (c) un informe mensual para el gerente.

---

3. Bloque 2 · Lectura y escritura de archivos de texto 🟢

¿Qué problema resuelve?

Guardar y recuperar información sencilla: listas de usuarios, historiales, **logs** de servidor, registros de incidencias.

3.1 La función `open()` y sus modos

```
archivo = open('datos/alumnos.txt', 'r', encoding='utf-8')
```

Modo	Operación	Comportamiento
`'r'`	Lectura (por defecto)	Error `FileNotFoundError` si no existe
`'w'`	Escritura	Crea el archivo; **si existe, lo sobrescribe y borra todo**
`'a'`	Añadir (*append*)	Conserva lo anterior y escribe al final
`'x'`	Creación exclusiva	Error si el archivo ya existe (evita borrados accidentales)

`encoding='utf-8'` es la codificación estándar de la web. Indícala **siempre**: sin ella, tildes, `ñ` y `€` pueden verse mal al cambiar de Windows a Linux.

3.2 Leer: de la forma clásica a `with`

Supón que `datos/alumnos.txt` contiene cuatro líneas: Carlos, Ana, Lucía y David.

```
# Forma clásica: hay que acordarse de cerrar
archivo = open('datos/alumnos.txt', 'r', encoding='utf-8')
contenido = archivo.read()
print(contenido)
archivo.close()

# Forma profesional: 'with' cierra el archivo solo, incluso si hay un error
with open('datos/alumnos.txt', 'r', encoding='utf-8') as archivo:
    contenido = archivo.read()
print(contenido)
```

`with` es un **gestor de contexto**: al salir del bloque, el recurso se libera automáticamente. Es el estándar en Python.

3.3 Leer línea a línea y `strip()`

Cada línea termina con un salto invisible (`\n`) y `print()` añade otro: por eso aparecen líneas en blanco. `strip()` elimina espacios y saltos de los extremos.

```
with open('datos/alumnos.txt', 'r', encoding='utf-8') as archivo:
    for linea in archivo:
        print(linea.strip())
```

Método	Qué devuelve
`read()`	Todo el contenido como un único `str`
`readline()`	Una línea por llamada
`readlines()`	Lista (`list[str]`) con todas las líneas
`for linea in archivo`	Línea a línea sin cargar todo en RAM (**ideal para archivos grandes**)

3.4 Escribir (`'w'`) y añadir (`'a'`)

```
# 'w': crea o sobrescribe
with open('datos/registro.txt', 'w', encoding='utf-8') as archivo:
    archivo.write('Inicio del sistema\n')
    archivo.write('Equipo: PC-01\n')

# 'a': conserva lo anterior y añade al final
with open('datos/registro.txt', 'a', encoding='utf-8') as archivo:
    archivo.write('Nuevo usuario conectado\n')
```

> `write()` **no añade el salto de línea**: tienes que incluir `\n` tú.

3.5 Ejemplo guiado: registro de incidencias con fecha y hora

El modo `'a'` es el idóneo para historiales y logs:

```
from datetime import datetime

incidencia = input('Introduce la incidencia: ')
fecha_hora = datetime.now().strftime('%Y-%m-%d %H:%M:%S')

with open('datos/incidencias.txt', 'a', encoding='utf-8') as archivo:
    archivo.write(f'[{fecha_hora}] {incidencia}\n')

print('Incidencia guardada correctamente')
```

🤖 Copilot en este bloque

- Pídele que te **explique** `with` con un ejemplo pequeño antes de usarlo.
- Audita siempre su código: ¿incluye `encoding='utf-8'`? ¿añade `\n` en cada `write()`? ¿usa `'w'` donde debería usar `'a'` (borraría datos)?

Prácticas

- 🟢 **Registro de usuarios.** Pide por consola nombre y departamento y guárdalos en `datos/usuarios.txt`, un usuario por línea (`Carlos;Sistemas`). Debe poder ejecutarse varias veces sin borrar los anteriores.
- 🟢 Crea un segundo programa que lea `usuarios.txt`, muestre los usuarios **numerados** (`1. Carlos - Sistemas`) e indique el total.
- 🟡 Usa `split(';')` para separar nombre y departamento y muestra cuántos usuarios hay en cada departamento (con un diccionario).
- 🔴 Crea un mini-logger: función `registrar(nivel, mensaje)` que escriba `[fecha] [INFO|ERROR] mensaje` en `resultados/app.log`. Después compáralo con el módulo estándar `logging` (lo usaremos en el Bloque 10).

Errores típicos: `FileNotFoundError` (ruta mal o VS Code abierto en otra carpeta); usar `'w'` y borrar datos sin querer; olvidar `\n`; no cerrar el archivo (por eso `with`); tildes rotas por falta de `encoding`.

---

4. Bloque 3 · Rutas y directorios con `pathlib` 🟢🟡

¿Qué problema resuelve?

Tu programa funciona en tu PC porque apunta a `C:\Users\Alumno\Desktop\...`, y falla en el de tu compañero o en un servidor Linux. `pathlib` gestiona rutas de forma **portable** (Windows usa `\`; Linux y macOS usan `/`) y con un estilo orientado a objetos.

4.1 Rutas absolutas y relativas

- **Absoluta:** ubicación completa desde la raíz del disco (`C:\Users\Alumno\proyecto\datos\alumnos.txt`). **Nunca** en proyectos compartidos: solo funciona en ese equipo.
- **Relativa:** toma como referencia una carpeta (`datos/alumnos.txt`). Permite que el programa funcione en cualquier equipo.

> **Matiz importante:** una ruta relativa se calcula desde el **directorio de trabajo** (desde donde ejecutas), no desde donde está el `.py`. Si abres VS Code en la carpeta equivocada, `datos/equipos.csv` no se encuentra. La solución profesional es anclar las rutas a la raíz del proyecto:

```
from pathlib import Path

BASE_DIR = Path(__file__).resolve().parent.parent   # raíz del proyecto
RUTA_CSV = BASE_DIR / 'datos' / 'equipos.csv'        # el operador / une
partes
print(RUTA_CSV)
```

`__file__` es la ruta del script en ejecución; `.parent.parent` sube de `ejercicios/` a la raíz.

4.2 Elementos esenciales de `Path`

```
from pathlib import Path

ruta = Path('datos/alumnos.txt')
print(ruta)          # datos/alumnos.txt
print(ruta.name)     # alumnos.txt
print(ruta.stem)     # alumnos
print(ruta.suffix)   # .txt
print(ruta.parent)   # datos
print(ruta.exists()) # True / False
```

Elemento	Función
`Path('a/b.txt')`	Crea un objeto ruta multiplataforma
`ruta / 'x.csv'`	Une partes de una ruta
`.name` / `.stem`	Nombre con extensión / sin extensión
`.suffix` / `.parent`	Extensión / carpeta contenedora
`.exists()`	`True` si existe en disco
`.is_file()` / `.is_dir()`	Comprueba si es archivo o carpeta
`.mkdir(exist_ok=True)`	Crea la carpeta sin error si ya existe
`.iterdir()`	Recorre el contenido de una carpeta
`.glob('*.csv')`	Busca por patrón en esa carpeta
`.rglob('*.csv')`	Igual, pero también en subcarpetas
`.stat().st_size`	Tamaño en bytes
`.read_text()` / `.write_text()`	Leer/escribir un archivo pequeño en una línea

4.3 Ejemplo guiado: explorador de archivos

```
from pathlib import Path

carpeta = Path('datos')
carpeta.mkdir(exist_ok=True)          # la crea si no existe

archivos = [a for a in carpeta.iterdir() if a.is_file()]

for archivo in sorted(archivos):
    kb = archivo.stat().st_size / 1024
    print(f'{archivo.name:25} {archivo.suffix:6} {kb:8.1f} KB')

print('Total de archivos:', len(archivos))

# Solo los CSV
for csv_file in carpeta.glob('*.csv'):
    print('CSV encontrado:', csv_file.name)
```

En `f'{archivo.name:25}'` el `:25` reserva 25 caracteres de ancho y `:8.1f` muestra un decimal: sirve para alinear columnas.

Para archivos pequeños hay un atajo:

```
Path('resultados/nota.txt').write_text('Hola\n', encoding='utf-8')
texto = Path('resultados/nota.txt').read_text(encoding='utf-8')
```

> **Contexto profesional:** las rutas y claves de una aplicación real no se escriben dentro del código, sino en un archivo `.env` (que no se sube a GitHub) que se lee con la librería `python-dotenv`. Lo veremos como ampliación.

🤖 Copilot en este bloque

Si Copilot te sugiere `os.path.join()` o `os.listdir()`, pídele: «*Refactoriza este código usando pathlib.Path*». Es el estándar moderno de Python 3. Comprueba además que no haya escrito rutas absolutas en el código.

Prácticas

- 🟢 **Explorador.** Comprueba si existe `datos/`; si no, créala. Muestra cada archivo con su nombre y extensión, y cuántos hay.
- 🟢 Muestra el tamaño de cada archivo en KB y cuál es el más grande.
- 🟡 Pide por teclado una extensión (`.csv`, `.json`) y lista solo esos archivos con `glob()`. Si no hay ninguno, informa al usuario.
- 🟡 Reescribe la práctica anterior con `BASE_DIR` para que funcione ejecutándola desde cualquier carpeta.
- 🔴 Crea una función `organizar_por_extension(origen)` que cree subcarpetas `csv/`, `json/`, `txt/` dentro de `resultados/` y **copie** (módulo `shutil`) cada archivo a la suya. Pide a Copilot que revise qué pasaría si un archivo ya existe.

Errores típicos: rutas absolutas del PC del alumno; olvidar que `mkdir()` falla si la carpeta existe (usa `exist_ok=True`); confundir `iterdir()` (una carpeta) con `rglob()` (recursivo); abrir VS Code en una carpeta superior.

---

5. Bloque 4 · Archivos CSV, funciones y programación defensiva 🟡

¿Qué problema resuelve?

Los datos tabulares (inventarios, listados de usuarios, exportaciones de una web) se intercambian constantemente en **CSV** (*Comma-Separated Values*): texto plano donde cada línea es un registro y cada valor separado por coma es un campo. Es el formato universal para pasar datos entre aplicaciones y Excel.

```
codigo,dispositivo,marca,modelo,ram,precio,departamento,estado
PC01,Sobremesa,Lenovo,ThinkCentre M70,16,650,Administración,Disponible
PC02,Sobremesa,HP,ProDesk 400,32,850,Sistemas,En uso
```

TXT	CSV
Texto libre, sin estructura obligatoria	Datos estructurados en campos y registros
Se procesa como líneas de texto	Se interpreta como una tabla
Ideal para notas y logs	Ideal para inventarios, catálogos y listados

5.1 Lectura: `csv.reader` y `csv.DictReader`

El módulo `csv` viene con Python. Al abrir un CSV se usa siempre `newline=''` para evitar saltos de línea duplicados.

```
import csv

# 1. Como listas
with open('datos/equipos.csv', 'r', encoding='utf-8', newline='') as archivo:
    lector = csv.reader(archivo)
    for fila in lector:
        print(fila)   # ['PC01', 'Sobremesa', 'Lenovo', ...] -> todo es texto

# 2. Como diccionarios (recomendado)
with open('datos/equipos.csv', 'r', encoding='utf-8', newline='') as archivo:
    lector = csv.DictReader(archivo)
    for equipo in lector:
        print(equipo['codigo'], equipo['modelo'], equipo['precio'])
```

`DictReader` usa la cabecera como claves (`{'codigo': 'PC01', 'precio': '650', ...}`), por lo que el código es más legible y no se rompe si cambia el orden de las columnas.

> **Trampa clásica:** `csv` lee **todo como texto**. `'650' > '700'` compara cadenas, no números (`'1100' < '700'` es `True`). Convierte siempre con `float()` o `int()` antes de operar.

5.2 Escritura: `writer` y `DictWriter`

```
import csv
from pathlib import Path

Path('resultados').mkdir(exist_ok=True)

equipos = [
    ['PC01', 'Lenovo ThinkCentre', 16, 650],
    ['PC02', 'HP ProDesk', 32, 850],
]

with open('resultados/equipos_nuevos.csv', 'w', encoding='utf-8', newline='')
as archivo:
    escritor = csv.writer(archivo)
    escritor.writerow(['codigo', 'modelo', 'ram', 'precio'])   # una fila
    escritor.writerows(equipos)                                # varias filas
```

Clase / método	Función
`csv.reader()`	Lee cada fila como lista
`csv.DictReader()`	Lee cada fila como diccionario
`csv.writer()` + `writerow()` / `writerows()`	Escribe filas desde listas
`csv.DictWriter()` + `writeheader()`	Escribe filas desde diccionarios

Dos detalles reales de España: Excel en configuración regional española exporta CSV con **punto y coma** como separador (`csv.reader(archivo, delimiter=';')`) y, a veces, con una marca BOM que se gestiona con `encoding='utf-8-sig'`.

5.3 Ejemplo guiado empresarial: funciones, Type Hints y `try/except`

En una empresa un CSV puede traer una fila vacía o un precio mal escrito (`'seiscientos'`). Si no protegemos la conversión, **el programa entero se detiene**. Por eso estructuramos el código en funciones pequeñas, con *type hints* (anotaciones de tipo: no se ejecutan, documentan) y captura de errores:

```
import csv
from pathlib import Path

CAMPOS = ['codigo', 'dispositivo', 'marca', 'modelo',
          'ram', 'precio', 'departamento', 'estado']
```

```
def leer_equipos(ruta: Path) -> list[dict]:
    """Lee el CSV y devuelve una lista de diccionarios (valores como
texto)."""
    with open(ruta, 'r', encoding='utf-8', newline='') as archivo:
        return list(csv.DictReader(archivo))


def equipos_caros(equipos: list[dict], limite: float) -> list[dict]:
    """Devuelve los equipos con precio superior al límite, avisando de datos
erróneos."""
    resultado = []
    for numero, equipo in enumerate(equipos, start=2):   # la línea 1 es la
cabecera
        try:
            precio = float(equipo['precio'])
        except (ValueError, KeyError, TypeError):
            print(f'Aviso línea {numero}: precio no válido')
            continue
        if precio > limite:
            resultado.append(equipo)
    return resultado


def anadir_equipo(ruta: Path, equipo: dict) -> None:
    """Añade un equipo al final del CSV sin borrar los anteriores."""
    es_nuevo = not ruta.exists()
    with open(ruta, 'a', encoding='utf-8', newline='') as archivo:
        escritor = csv.DictWriter(archivo, fieldnames=CAMPOS)
        if es_nuevo:
            escritor.writeheader()
        escritor.writerow(equipo)


if __name__ == '__main__':
    equipos = leer_equipos(Path('datos/equipos.csv'))
    for e in equipos_caros(equipos, 700):
        print(e['codigo'], e['modelo'], e['precio'], '€')
```

Qué debes ser capaz de explicar: por qué `newline=''`; qué hace `enumerate(..., start=2)`; por qué capturamos `ValueError` (texto no numérico) y `KeyError` (columna que no existe); por qué `anadir_equipo` usa `'a'`; y para qué sirve `if __name__ == '__main__':` (el código de abajo solo se ejecuta si lanzas el archivo directamente, no si lo importas).

🤖 Copilot en este bloque

- Pídele que **explique** una línea que no entiendas («*¿qué hace* `enumerate(equipos, start=2)` *?*»), no que escriba la función.
- Auditoría: ¿convierte los precios a número? ¿incluye `newline=''` y `encoding`? ¿usa `'w'` en lugar de `'a'` al añadir? ¿captura `except:` genérico sin indicar el error (mala práctica)?
- Tras escribir tu función, usa `/tests` para que proponga casos con datos incorrectos (precio vacío, texto, columna ausente).

Prácticas

- 🟢 **Lectura.** Con `DictReader`, muestra los equipos cuyo `estado` sea `Disponible`.
- 🟢 Muestra los equipos con precio superior a 500 € convirtiendo con `float()`.
- 🟢 Calcula el valor económico total del inventario.
- 🟡 **Escritura.** Permite añadir un equipo pedido por teclado al final de `equipos.csv` sin borrar los anteriores (`DictWriter` + modo `'a'`). Valida que el precio sea numérico y que el código no esté repetido.
- 🟡 Reorganiza todo en funciones con *type hints* y prueba con un CSV que contenga una fila con precio vacío y otra con texto.
- 🟡 Busca un equipo por código introducido por teclado y muestra todos sus datos (o un mensaje si no existe).
- 🔴 **Mini-aplicación de consola** con menú (listar, buscar, añadir, dar de baja cambiando el estado a `Baja`). Para modificar un registro, léelo todo, cambia el diccionario y reescribe el archivo completo. Entrega con commits por funcionalidad.

Errores típicos: comparar precios como texto; olvidar `newline=''`; escribir con `'w'` y perder el inventario; `try/except` que oculta el error (`except: pass`); cabeceras que no coinciden con `fieldnames`; separador `;` no detectado.

---

6. Bloque 5 · JSON y consumo de APIs web 🟡

¿Qué problema resuelve?

Cuando tu frontend (JavaScript) habla con tu backend (Python), o tu aplicación consume una API de terceros, el formato de intercambio estándar es **JSON** (*JavaScript Object Notation*). A diferencia de CSV, admite **estructuras jerárquicas**: objetos dentro de objetos y listas dentro de objetos.

```
{
    "codigo": "PC01",
    "modelo": "Lenovo ThinkCentre",
    "ram": 16,
    "precio": 650,
    "disponible": true
}
```

6.1 Equivalencia JSON ↔ Python

JSON	Python	Nota
Objeto `{}`	`dict`	Las claves van **siempre** entre comillas dobles
Array `[]`	`list`	Puede contener números, textos u otros objetos
String	`str`	Comillas dobles (JSON no admite simples)
Number	`int` / `float`	Se convierte directamente a número
`true` / `false`	`True` / `False`	Minúscula en JSON, mayúscula inicial en Python
`null`	`None`	JSON **no admite comentarios**

6.2 Leer y escribir JSON

```
import json

# Leer: JSON -> estructuras de Python
with open('datos/inventario.json', 'r', encoding='utf-8') as archivo:
    datos = json.load(archivo)

for equipo in datos['equipos']:
    print(equipo['codigo'], '-', equipo['ram'], 'GB')

# Escribir: Python -> JSON
nuevo = {'codigo': 'PC11', 'modelo': 'HP ProDesk', 'ram': 32,
         'precio': 850, 'disponible': True}

with open('resultados/equipo.json', 'w', encoding='utf-8') as archivo:
    json.dump(nuevo, archivo, indent=4, ensure_ascii=False)
```

Función / argumento	Qué hace
`json.load(archivo)` / `json.dump(obj, archivo)`	Leer / escribir desde/hacia un **archivo**
`json.loads(texto)` / `json.dumps(obj)`	Convertir desde/hacia una **cadena** (la `s` es de *string*)
`indent=4`	Sangría legible
`ensure_ascii=False`	Conserva `ñ`, tildes y `€`

Estructura habitual de `datos/inventario.json` (lista de objetos dentro de un objeto):

```
{
    "equipos": [
        {"codigo": "PC01", "ram": 16, "precio": 650},
        {"codigo": "PC02", "ram": 32, "precio": 850}
    ]
}
```

6.3 Patrón fundamental: leer → modificar → escribir

JSON no permite «añadir al final» como un log: se **carga todo, se modifica en memoria y se reescribe**. Es el patrón que usarás para cualquier persistencia sencilla.

```
import json
from pathlib import Path

RUTA = Path('datos/usuarios.json')


def cargar_usuarios(ruta: Path) -> list[dict]:
    try:
        with open(ruta, 'r', encoding='utf-8') as archivo:
            return json.load(archivo)['usuarios']
    except FileNotFoundError:
        return []                                   # primera ejecución: aún
no existe
    except json.JSONDecodeError as error:
        print(f'JSON mal formado: {error}')
        return []


def guardar_usuarios(ruta: Path, usuarios: list[dict]) -> None:
    with open(ruta, 'w', encoding='utf-8') as archivo:
```

```
        json.dump({'usuarios': usuarios}, archivo, indent=4,
ensure_ascii=False)


usuarios = cargar_usuarios(RUTA)
usuarios.append({'nombre': 'Lucía Pérez', 'departamento': 'Ventas',
                 'correo': 'lucia@empresa.local', 'activo': True})
guardar_usuarios(RUTA, usuarios)

activos = [u for u in usuarios if u['activo']]
print('Usuarios activos:', len(activos))
```

6.4 Enfoque DAW: consumir una API REST con `requests`

En desarrollo web, gran parte del JSON llega por **HTTP**. La librería `requests` descarga la respuesta y la convierte en estructuras de Python.

```
import json
import requests

URL = 'https://jsonplaceholder.typicode.com/users'   # API pública de pruebas


def descargar_usuarios(url: str) -> list[dict]:
    try:
        respuesta = requests.get(url, timeout=5)   # SIEMPRE con timeout
        respuesta.raise_for_status()               # lanza error si el código
es 4xx/5xx
        return respuesta.json()                    # JSON -> lista de Python
    except requests.RequestException as error:
        print(f'No se pudo contactar con la API: {error}')
        return []


usuarios = descargar_usuarios(URL)
with open('datos/usuarios_remotos.json', 'w', encoding='utf-8') as archivo:
    json.dump(usuarios, archivo, indent=4, ensure_ascii=False)
print(f'Guardados {len(usuarios)} usuarios')
```

Código HTTP	Significado
200	Correcto
404	Recurso no encontrado

Código HTTP	Significado
401 / 403	No autenticado / sin permiso
500	Error en el servidor

Buenas prácticas de empresa: usa siempre `timeout`; gestiona el fallo de red (puede no haber internet en el aula: ten un JSON de respaldo en `datos/`); **nunca** escribas claves de API en el código ni las subas a GitHub: se guardan en `.env` y se leen con `os.environ.get('API_KEY')`.

6.5 CSV frente a JSON

Característica	CSV	JSON
Estructura	Tabla plana (filas y columnas)	Objetos y listas jerárquicos
Datos anidados	No de forma natural	Sí
Se abre directamente en Excel	Sí	No
Tipos de dato	Todo es texto	Números, booleanos, `null`
Uso típico	Exportaciones, inventarios	APIs REST, configuración

🤖 Copilot en este bloque

- Pídele que **valide** un JSON que no carga («*¿por qué `json.load` da `JSONDecodeError`?*»): suele ser una coma sobrante o comillas simples.
- Auditoría: ¿usa `timeout`? ¿gestiona `RequestException`? ¿mantiene `ensure_ascii=False`? ¿ha dejado alguna clave o URL privada escrita en el código?

Prácticas

- 🟢 **Usuarios JSON.** Crea `datos/usuarios.json` con 4 usuarios (`nombre`, `departamento`, `correo`, `activo`). Léelo y muestra solo los activos.
- 🟢 Añade un usuario nuevo desde Python y guarda el archivo manteniendo `indent=4` y `ensure_ascii=False`. Comprueba en VS Code que el JSON sigue siendo válido.
- 🟡 Convierte `equipos.csv` en `inventario.json` (lista de objetos con `ram` y `precio` como **números**, no texto).

- 🟡 Descarga usuarios de la API de pruebas y muestra nombre, correo y ciudad (`usuario['address']['city']`: datos anidados). Si no hay red, usa el JSON de respaldo.
- 🔴 **Menú interactivo:** consultar, añadir, desactivar (`activo = False`) e importar usuarios desde la API, evitando duplicados por correo. Usa funciones y *type hints*.

Errores típicos: comillas simples o coma final en el JSON; olvidar que `true` es `True` en Python; usar `json.load` con una cadena (es `json.loads`); abrir con `'w'` antes de leer (borra el archivo); no usar `timeout`.

---

7. Bloque 6 · Introducción a pandas y DataFrames 🟡

¿Qué problema resuelve?

Con `csv` y bucles `for` puedes procesar un inventario de 10 filas, pero filtrar, agrupar o calcular estadísticas sobre 100.000 filas se vuelve lento y largo de escribir. **pandas** es la librería de referencia para trabajar con datos tabulares: leer archivos, filtrar, transformar, calcular métricas y exportar resultados con muy pocas líneas.

```
python -m pip install pandas openpyxl
```

```
import pandas as pd     # alias estándar
```

7.1 DataFrame y Series

- **DataFrame:** tabla bidimensional de filas y columnas (como una hoja de cálculo o una tabla SQL). Cada columna tiene su propio tipo de dato.
- **Series:** una sola columna de un DataFrame.
- **Índice:** etiqueta de cada fila (por defecto 0, 1, 2…).

```
import pandas as pd

datos = {
    'codigo': ['PC01', 'PC02', 'PC03'],
    'modelo': ['Lenovo', 'HP', 'Dell'],
    'ram': [16, 32, 16],
    'precio': [650, 850, 720],
}
df = pd.DataFrame(datos)     # cada clave es una columna; cada lista, sus
valores
print(df)
```

7.2 Inspeccionar un DataFrame

```
df = pd.read_csv('datos/equipos.csv')

print(df.head())        # primeras 5 filas
print(df.tail())        # últimas 5 filas
print(df.shape)         # (filas, columnas)
print(df.columns)       # nombres de columnas
print(df.dtypes)        # tipo de cada columna
df.info()               # estructura, tipos y nulos (¡sin print!)
print(df.describe())    # estadísticas de las columnas numéricas
```

> `df.info()` **imprime por sí mismo** y devuelve `None`; si escribes `print(df.info())` verás un `None` sobrante.

7.3 Seleccionar columnas y filas

```
print(df['modelo'])                 # una columna (Series)
print(df[['codigo', 'precio']])     # varias columnas: doble corchete ->
DataFrame

print(df.loc[0])        # fila por ETIQUETA de índice
print(df.iloc[1])       # fila por POSICIÓN
print(df.iloc[0:2])     # posiciones 0 y 1 (el final no se incluye)
```

7.4 Operaciones vectorizadas (sin bucles)

```
print('Total:', df['precio'].sum())
print('Media:', round(df['precio'].mean(), 2))
print('Máximo:', df['precio'].max())
print('Mínimo:', df['precio'].min())
print('Equipos:', len(df))

# El equipo más caro y el más barato (fila completa)
print(df.loc[df['precio'].idxmax()])
print(df.loc[df['precio'].idxmin()])

# Nueva columna calculada para TODAS las filas a la vez
df['precio_iva'] = (df['precio'] * 1.21).round(2)

# Contar categorías
print(df['estado'].value_counts())
```

Vectorizar significa aplicar una operación a toda la columna de una vez. Es más corto, más legible y mucho más rápido que recorrer fila a fila con un `for`.

7.5 Leer y guardar archivos

```
df = pd.read_csv('datos/equipos.csv')                          # CSV estándar
df = pd.read_csv('datos/equipos_es.csv', sep=';')              # separador punto
y coma
df = pd.read_csv('datos/equipos.csv', encoding='utf-8')

df.to_csv('resultados/equipos_iva.csv', index=False)           # index=False:
sin columna 0,1,2...

# De JSON a DataFrame
import json
with open('datos/usuarios.json', 'r', encoding='utf-8') as archivo:
    usuarios = json.load(archivo)['usuarios']
df_usuarios = pd.DataFrame(usuarios)
# Para JSON con datos anidados: pd.json_normalize(lista_de_dicts)
```

7.6 pandas frente a `csv`: ¿cuándo usar cada uno?

Situación	Herramienta
Leer 20 filas y mostrarlas	`csv` basta
Filtrar, agrupar, calcular estadísticas	pandas
Fichero enorme que no cabe en RAM	`csv` línea a línea o pandas con `chunksize`
Limpiar datos sucios	pandas
Generar informes Excel	pandas + openpyxl

🤖 Copilot en este bloque

- Pídele: «*Explícame la diferencia entre `loc` e `iloc` con un ejemplo de 3 filas*».
- Comprueba si te propone un bucle `for` para algo que pandas resuelve en una línea (`df['precio'].sum()`); pídele la versión vectorizada.
- Antes de aceptar una cadena de métodos larga, pídele que la **divida en pasos** y explique cada uno.

Prácticas

- 🟢 **Primer análisis.** Lee `equipos.csv` y muestra las 5 primeras filas, el número de equipos, el precio medio, el equipo más caro y el más barato.
- 🟢 Añade la columna `precio_iva` (×1,21, redondeada a 2 decimales) y expórtala a `resultados/equipos_iva.csv`.
- 🟡 Muestra cuántos equipos hay por estado y por marca (`value_counts()`), y la RAM media del parque.
- 🟡 Carga `usuarios.json` en un DataFrame y exporta solo `nombre` y `correo` de los activos a CSV.
- 🔴 Genera `resultados/resumen.txt` con número de equipos, precio medio, valor total y el equipo más caro. Después escribe la misma lógica con un bucle `for` y con pandas, y mide el tiempo con 100.000 filas generadas (`time.perf_counter()`).

Errores típicos: `print(df.info())`; usar un corchete (`df['a','b']`) en lugar de dos (`df[['a','b']]`); olvidar `index=False`; confundir `loc` con `iloc`; trabajar con la ruta equivocada en `read_csv`; olvidar instalar pandas en el `.venv` correcto.

---

8. Bloque 7 · Excel con pandas y openpyxl 🟡

¿Qué problema resuelve?

En la empresa, los informes de gestión se entregan casi siempre en **Excel (`.xlsx`)**. Con Python puedes leer hojas que te envía otro departamento y generar libros con varias pestañas de forma automática cada mes, sin copiar y pegar.

- **pandas** lee y escribe tablas (`read_excel`, `to_excel`).
- **openpyxl** es el motor que maneja el formato `.xlsx` y permite tocar celdas, estilos y fórmulas.

8.1 Leer y escribir

```
import pandas as pd

# Leer una hoja concreta (por nombre o por posición)
df = pd.read_excel('datos/inventario.xlsx', sheet_name='Hoja1')

# Leer TODAS las hojas: devuelve un diccionario {nombre_hoja: DataFrame}
hojas = pd.read_excel('datos/inventario.xlsx', sheet_name=None)

# Exportar a un Excel sencillo
df.to_excel('resultados/inventario_exportado.xlsx', index=False)
```

8.2 Libro con varias hojas y formato profesional

```
import pandas as pd
from openpyxl.styles import Font, PatternFill

df = pd.read_csv('datos/equipos.csv')
resumen = df.groupby('departamento')['precio'].agg(['count', 'sum',
'mean']).round(2)

with pd.ExcelWriter('resultados/informe.xlsx', engine='openpyxl') as writer:
    df.to_excel(writer, sheet_name='Inventario', index=False)
    resumen.to_excel(writer, sheet_name='Resumen')

    hoja = writer.sheets['Inventario']            # hoja de openpyxl para dar
formato

    for celda in hoja[1]:                         # fila 1: cabeceras
        celda.font = Font(bold=True, color='FFFFFF')
        celda.fill = PatternFill('solid', fgColor='1F4E78')

    hoja.freeze_panes = 'A2'                      # inmoviliza la cabecera
    hoja.auto_filter.ref = hoja.dimensions        # filtros desplegables

    for columna in hoja.columns:                  # ancho de columna
automático
        ancho = max(len(str(c.value)) if c.value is not None else 0 for c in
columna)
        hoja.column_dimensions[columna[0].column_letter].width = ancho + 3
```

```
    for fila in hoja.iter_rows(min_row=2, min_col=6, max_col=6):   # columna
precio
        for celda in fila:
            celda.number_format = '#,##0.00 €'
```

Matices útiles:

- pandas escribe **valores**, no fórmulas. Si el informe debe recalcularse en Excel, escribe la fórmula con openpyxl: `hoja['H12'] = '=SUM(F2:F11)'`.
- `to_excel` sin `index=False` añade una columna con 0, 1, 2…; en un `groupby` el índice (el departamento) **sí** interesa, por eso en `Resumen` no lo quitamos.
- Los ficheros `.xls` antiguos y los `.xlsm` con macros requieren cuidado: no abras ficheros de origen desconocido.

🤖 Copilot en este bloque

Pídele que añada formato (*«pon la cabecera en negrita y azul»*), pero **verifica abriendo el `.xlsx`** en Excel o LibreOffice: Copilot a veces inventa atributos de openpyxl que no existen. Comprueba también que no se pierdan las tildes ni se sobrescriba un informe anterior sin querer.

Prácticas

- 🟢 Lee `datos/inventario.xlsx`, filtra los equipos `Disponible` y calcula su valor total.
- 🟢 Genera `resultados/informe.xlsx` con dos hojas: `Inventario` y `Resumen`.
- 🟡 Da formato a la cabecera, inmoviliza la primera fila y muestra el precio con el símbolo €.
- 🔴 Añade una tercera hoja `Incidencias` y una fila final de totales con una **fórmula** de Excel. Después ábrelo con `pd.read_excel` y comprueba si pandas lee la fórmula o su resultado (y explica por qué).

Errores típicos: no tener `openpyxl` instalado (`ImportError`); intentar escribir en un Excel abierto en otro programa (`PermissionError`); nombre de hoja con más de 31 caracteres; olvidar `index=False`.

---

9. Bloque 8 · Limpieza y transformación de datos («datos sucios») 🟡🔴

¿Qué problema resuelve?

Los datos reales **nunca llegan limpios**: celdas vacías, duplicados, espacios sobrantes, `HP` / `hp` / `Hp`, precios escritos como `abc`. Si analizas sin limpiar, los resultados serán falsos. La regla de oro: **inspeccionar → decidir → transformar → registrar lo que se descarta**. Y nunca modifiques el archivo original: la limpieza genera un archivo nuevo.

Crea `datos/equipos_sucio.csv` con este contenido:

```
codigo,dispositivo,marca,modelo,ram,precio,departamento,estado
PC01,Sobremesa, lenovo ,ThinkCentre M70,16,650,Administración,Disponible
PC02,Sobremesa,HP,ProDesk 400,32,850,Sistemas,En uso
PC02,Sobremesa,HP,ProDesk 400,32,850,Sistemas,En uso
PC03,Portátil,DELL,Latitude 5440,16,abc,Ventas,En uso
PC04,Portátil,Lenovo,ThinkPad E14,,780,Ventas,En uso
,Portátil,hp,ProBook 450,32,990,Sistemas,En uso
PC06,Sobremesa,dell ,OptiPlex 7010,16,720,administración,disponible
PC07,Portátil,Asus,ExpertBook B1,8,,Administración,En uso
```

9.1 Catálogo de problemas y cómo tratarlos

Problema	Cómo detectarlo	Tratamiento habitual
Valores nulos (`NaN`)	`df.isna().sum()`	Eliminar la fila, marcarla o imputar **solo si se justifica**
Duplicados	`df.duplicated(subset=['codigo']).sum()`	`drop_duplicates(subset=['codigo'], keep='first')`
Espacios y mayúsculas	`df['marca'].unique()`	`str.strip()`, `str.lower()`, `map()`
Tipos incorrectos	`df.dtypes`	`pd.to_numeric(..., errors='coerce')`
Valores imposibles (precio negativo, RAM 0)	`df.describe()`	Filtrar y registrar la incidencia
Categorías inconsistentes (`en uso`, `En Uso`)	`df['estado'].value_counts()`	Normalizar con `str.capitalize()`

> **Decidir bien es parte del trabajo.** No sustituyas todos los nulos por 0: *un precio desconocido no significa que sea gratis*. Rellenar la RAM con un valor «típico» (por ejemplo 16 GB) inventa un dato que no existe; si lo haces, que sea una decisión **documentada** (media, mediana o «desconocido»), nunca automática. Muchas veces lo correcto es excluir la fila y anotarla en un informe de incidencias.

9.2 Inspeccionar primero

```
import pandas as pd

df = pd.read_csv('datos/equipos_sucio.csv')

df.info()
print(df.isna().sum())                                # nulos por columna
print(df[df.isna().any(axis=1)])                      # filas con algún nulo
print('Duplicados:', df.duplicated(subset=['codigo']).sum())
print(df['marca'].unique())                           # ¿hay variantes de una
misma marca?
print(df['estado'].value_counts())
```

9.3 Transformar paso a paso

```
MARCAS = {'hp': 'HP', 'dell': 'Dell', 'lenovo': 'Lenovo', 'asus': 'Asus'}


def limpiar_equipos(df: pd.DataFrame) -> tuple[pd.DataFrame, pd.DataFrame]:
    """Devuelve (equipos_limpios, equipos_descartados)."""
    df = df.copy()                                      # no tocamos el
original

    # 1. Textos: quitar espacios y vacíos
    for col in ['codigo', 'dispositivo', 'marca', 'modelo', 'departamento',
'estado']:
        df[col] = df[col].astype('string').str.strip().replace('', pd.NA)

    # 2. Normalizar categorías
    marca_norm = df['marca'].str.lower().map(MARCAS)
    df['marca'] = marca_norm.fillna(df['marca'].str.title())   # str.title()
convertiría HP en 'Hp'
    df['departamento'] = df['departamento'].str.capitalize()
    df['estado'] = df['estado'].str.capitalize()

    # 3. Tipos numéricos: lo no convertible pasa a NaN (en lugar de romper)
    df['precio'] = pd.to_numeric(df['precio'], errors='coerce')
```

```
    df['ram'] = pd.to_numeric(df['ram'], errors='coerce').astype('Int64')

    # 4. Registros irrecuperables: sin código o sin precio
    invalidos = df['codigo'].isna() | df['precio'].isna()
    descartados = df[invalidos].assign(motivo='Sin código o precio no
válido')
    limpios = df[~invalidos]

    # 5. Duplicados por código
    limpios = limpios.drop_duplicates(subset=['codigo'], keep='first')

    return limpios.reset_index(drop=True), descartados


limpios, descartados =
limpiar_equipos(pd.read_csv('datos/equipos_sucio.csv'))
limpios.to_csv('resultados/equipos_limpio.csv', index=False)
descartados.to_csv('resultados/incidencias.csv', index=False)
print(f'Limpios: {len(limpios)} | Descartados: {len(descartados)}')
```

Para debatir en clase: ¿por qué usamos `.copy()`? (para evitar modificar una vista y el aviso `SettingWithCopyWarning`). ¿Por qué `errors='coerce'` y no dejar que falle? ¿Qué hacemos con PC04, que tiene precio pero no RAM? (se conserva: la RAM es un dato complementario y `Int64` admite nulos).

🤖 Copilot en este bloque

- Pídele «*revisa mi función de limpieza y dime qué casos sucios no contempla*», no «*límpiame este CSV*».
- Auditoría: ¿propone `fillna(0)` para precios? ¿usa `str.title()` y rompe siglas como `HP`? ¿elimina filas sin dejar constancia de ello?
- Usa `/tests` para generar un DataFrame sucio de prueba y comprueba las filas esperadas.

Prácticas

- 🟢 Con `equipos_sucio.csv`: cuenta duplicados y nulos por columna.
- 🟢 Normaliza `marca` (sin espacios y con mayúsculas correctas) y convierte `precio` a número.
- 🟡 Elimina los registros irrecuperables y exporta `equipos_limpio.csv`.
- 🟡 Genera un informe de incidencias (`incidencias.csv`) con el código, el motivo y la fila original.

- 🔴 Añade una regla que descarte precios negativos o RAM fuera de `{4, 8, 16, 32, 64, 128}` y escribe pruebas con `pytest` para tres casos sucios.

Errores típicos: modificar el CSV original; `fillna(0)` indiscriminado; olvidar `.copy()`; usar `str.title()` con siglas; olvidar que `NaN` hace que una columna entera sea `float`; no hacer `reset_index(drop=True)` tras filtrar.

---

10. Bloque 9 · Filtrado, agrupación y agregación 🔴

¿Qué problema resuelve?

Una vez limpios los datos, se responden **preguntas de negocio**: ¿cuánto vale el parque por departamento?, ¿qué marca tiene el precio medio más alto?, ¿cuántos equipos están en reparación? Es la base de cualquier consulta analítica, informe o *dashboard*.

10.1 Filtrado booleano

Una condición como `df['precio'] > 700` produce una **máscara**: una serie de `True` / `False`, una por fila. Al usarla entre corchetes, pandas se queda con las filas `True`.

```
df = pd.read_csv('datos/equipos.csv')

caros = df[df['precio'] > 700]                                    # 8 equipos
con_32 = df[df['ram'] == 32]
filtro = df[(df['ram'] == 32) & (df['precio'] < 1000)]            # PC02, PC05
y PC07
no_hp = df[~(df['marca'] == 'HP')]
de_sistemas_o_ventas = df[df['departamento'].isin(['Sistemas', 'Ventas'])]
lenovo_thinkpad = df[df['modelo'].str.contains('ThinkPad', na=False)]
rango = df[df['precio'].between(600, 900)]
igual_que_antes = df.query('ram == 32 and precio < 1000')         # sintaxis
alternativa
```

Operador	Significado	Ejemplo
`==` / `!=`	Igual / distinto	`df[df['ram'] == 32]`
`>` `<` `>=` `<=`	Comparación	`df[df['precio'] > 700]`
`&`	Y lógico	`df[(c1) & (c2)]`
`|`	O lógico	`df[(c1) | (c2)]`
`~`	Negación	`df[~(c1)]`

> En pandas **no** se usan `and`, `or`, `not`. Y cada condición va **entre paréntesis** al combinarlas con `&` o `|`; si no, obtendrás un `ValueError`.

10.2 Ordenar y quedarse con los mejores

```
df.sort_values('precio', ascending=False)                        # de mayor a
menor
df.sort_values(['departamento', 'precio'], ascending=[True, False])
df.nlargest(3, 'precio')                                         # los 3 más
caros
```

10.3 Agrupar y agregar (`groupby` + `agg`)

`groupby` aplica tres pasos: **dividir** los datos por categoría, **calcular** una métrica en cada grupo y **combinar** los resultados.

```
# Valor del inventario por departamento
print(df.groupby('departamento')['precio'].sum())

# Varias métricas a la vez
resumen = df.groupby('departamento')['precio'].agg(['count', 'sum', 'mean',
'min', 'max'])
print(resumen)

# Versión profesional: agregación con nombres de columna claros
resumen = (
    df.groupby('departamento')
      .agg(equipos=('codigo', 'count'),
           valor_total=('precio', 'sum'),
           precio_medio=('precio', 'mean'))
      .round(2)
      .reset_index()
)
```

Con el inventario de 10 equipos del curso, el resultado esperado es:

departamento	count	sum	mean	min	max
Administración	3	1910	636,67	540	720
Dirección	2	2350	1175,00	1100	1250
Sistemas	3	2750	916,67	850	990
Ventas	2	1650	825,00	780	870

Verifica estos valores ejecutando tu propio código: el resultado de pandas debe coincidir con la tabla.

Otras agrupaciones útiles:

```
df.groupby('marca')['precio'].mean().round(2)          # precio medio por
marca
df['estado'].value_counts()                            # equipos por estado
df.groupby(['departamento', 'estado']).size()          # recuento por pareja

# Tabla cruzada tipo Excel
pd.pivot_table(df, index='departamento', columns='estado',
               values='codigo', aggfunc='count', fill_value=0)

# Filtrar GRUPOS: departamentos con inventario superior a un umbral
grandes = resumen[resumen['valor_total'] > 2000]
```

Función	Resultado
`count`	Número de registros
`sum`	Suma
`mean`	Media
`min` / `max`	Mínimo / máximo
`nunique`	Valores distintos

10.4 Combinar tablas: `merge` (lo que en SQL es un `JOIN`)

Imagina un JSON con los responsables de cada departamento:

```
{
    "responsables": [
        {"departamento": "Sistemas", "responsable": "Marta Gil"},
        {"departamento": "Ventas", "responsable": "Iván Ríos"}
    ]
}
```

```
import json
import pandas as pd

with open('datos/responsables.json', 'r', encoding='utf-8') as archivo:
    responsables = pd.DataFrame(json.load(archivo)['responsables'])

resumen = resumen.merge(responsables, on='departamento', how='left')   #
left: conserva todos los departamentos
```

🤖 Copilot en este bloque

- Describe la pregunta de negocio y pídele solo la **explicación** («*¿qué hace* `groupby().agg()` *paso a paso?*»), después escribe tu versión.
- Auditoría: ¿pone paréntesis en las condiciones? ¿usa `and` en lugar de `&`? ¿calcula la media sobre una columna que aún es texto? ¿devuelve el mismo resultado que tu cálculo manual con 3 o 4 filas?

Prácticas

- 🟢 Muestra los equipos de una marca introducida por teclado y los de más de 16 GB de RAM.
- 🟢 Ordénalos por precio descendente y muestra los 3 más caros.
- 🟡 Calcula el valor total por departamento, el precio medio por marca y el número de equipos por estado.
- 🟡 Exporta el resumen a CSV y a un Excel con una hoja por cada análisis.
- 🔴 Con `equipos_grande.csv` (80 equipos, ver Anexo C) identifica los departamentos con inventario superior a 5.000 €, añade el responsable mediante `merge` y genera el informe.
- 🔴 Crea una tabla cruzada departamento × estado y localiza el departamento con más equipos en reparación.

Errores típicos: `and` / `or` en lugar de `&` / `|`; falta de paréntesis; agrupar por una columna con variantes sin limpiar (`HP` y `hp` salen como grupos distintos); calcular la media sobre texto; `groupby` sin `reset_index()` cuando luego quieres tratar el resultado como tabla normal.

---

11. Bloque 10 · Proyecto integrador: mini ETL 🔴

¿Qué problema resuelve?

En una empresa, los datos llegan en bruto desde varios orígenes y alguien debe convertirlos en información fiable. Ese proceso se llama **ETL**: **E**xtract (extraer), **T**ransform (transformar) y **L**oad (cargar/exportar). Este proyecto reúne todo lo que has aprendido en la unidad y se entrega como **repositorio reproducible en GitHub**.

11.1 Enunciado

La empresa recibe cada mes `datos/inventario_bruto.csv` (con duplicados, precios corruptos, espacios y marcas mal escritas) y `datos/responsables.json` (responsable de cada departamento). Debes construir un programa que genere:

- `resultados/inventario_web.csv` y `resultados/inventario_web.json`: datos limpios, listos para servir desde una API o una aplicación web.
- `resultados/reporte_direccion.xlsx`: informe ejecutivo con **tres pestañas** (`Inventario`, `Por departamento`, `Por estado`).
- `resultados/incidencias.csv` y `resultados/etl.log`: filas descartadas y registro de lo ocurrido, con fecha y hora.

Fase	Tareas
Extract	Comprobar con `pathlib` que existen los archivos; leer el CSV con pandas y el JSON de responsables; registrar la extracción en el log
Transform	Limpiar duplicados, nulos, textos y tipos; descartar registros irrecuperables anotándolos; calcular `precio_iva`; generar resúmenes con `groupby` + `agg`; unir los responsables con `merge`
Load	Exportar CSV, JSON y Excel con varias hojas; guardar incidencias
Validación	Probar con datos correctos y sucios; ejecutar `pytest`; explicar el código y registrar cambios en Git

11.2 Estructura del repositorio

```
proyecto_etl/
├── datos/
|   ├── inventario_bruto.csv
|   └── responsables.json
├── resultados/            # se genera al ejecutar (ignorado por Git)
├── tests/
|   └── test_limpieza.py
├── main.py
├── requirements.txt
├── .gitignore
└── README.md
```

11.3 Esqueleto de `main.py` (completa las partes marcadas)

```
import json
import logging
from pathlib import Path

import pandas as pd

BASE_DIR = Path(__file__).resolve().parent
DATOS = BASE_DIR / 'datos'
RESULTADOS = BASE_DIR / 'resultados'
IVA = 0.21


def configurar_logging() -> None:
    RESULTADOS.mkdir(exist_ok=True)
    logging.basicConfig(
        filename=RESULTADOS / 'etl.log',
        level=logging.INFO,
        format='%(asctime)s [%(levelname)s] %(message)s',
        encoding='utf-8',
    )


# ---------- EXTRACT ----------
def extraer(ruta_csv: Path, ruta_json: Path) -> tuple[pd.DataFrame,
pd.DataFrame]:
    for ruta in (ruta_csv, ruta_json):
        if not ruta.exists():
            raise FileNotFoundError(f'No existe el archivo: {ruta}')
    inventario = pd.read_csv(ruta_csv)
    with open(ruta_json, 'r', encoding='utf-8') as archivo:
```

```
        responsables = pd.DataFrame(json.load(archivo)['responsables'])
    logging.info('Extraídos %d equipos y %d responsables', len(inventario),
len(responsables))
    return inventario, responsables


# ---------- TRANSFORM ----------
def limpiar_equipos(df: pd.DataFrame) -> tuple[pd.DataFrame, pd.DataFrame]:
    # TODO: reutiliza y revisa la función del Bloque 8
    ...


def anadir_iva(df: pd.DataFrame) -> pd.DataFrame:
    df = df.copy()
    df['precio_iva'] = (df['precio'] * (1 + IVA)).round(2)
    return df


def crear_resumenes(df: pd.DataFrame, responsables: pd.DataFrame) ->
dict[str, pd.DataFrame]:
    por_departamento = (
        df.groupby('departamento')
          .agg(equipos=('codigo', 'count'),
               valor_total=('precio', 'sum'),
               valor_con_iva=('precio_iva', 'sum'))
          .round(2)
          .reset_index()
          .merge(responsables, on='departamento', how='left')
    )
    por_estado =
df['estado'].value_counts().rename_axis('estado').reset_index(name='equipos')
    return {'Por departamento': por_departamento, 'Por estado': por_estado}


# ---------- LOAD ----------
def cargar(limpios: pd.DataFrame, descartados: pd.DataFrame,
           resumenes: dict[str, pd.DataFrame]) -> None:
    limpios.to_csv(RESULTADOS / 'inventario_web.csv', index=False)
    limpios.to_json(RESULTADOS / 'inventario_web.json',
                    orient='records', indent=4, force_ascii=False)
    descartados.to_csv(RESULTADOS / 'incidencias.csv', index=False)

    with pd.ExcelWriter(RESULTADOS / 'reporte_direccion.xlsx',
engine='openpyxl') as writer:
        limpios.to_excel(writer, sheet_name='Inventario', index=False)
        for nombre, tabla in resumenes.items():
```

```
            tabla.to_excel(writer, sheet_name=nombre, index=False)
        # TODO: formato de cabeceras y precios con openpyxl
    logging.info('Carga completada: %d equipos limpios', len(limpios))


def main() -> None:
    configurar_logging()
    try:
        bruto, responsables = extraer(DATOS / 'inventario_bruto.csv',
                                      DATOS / 'responsables.json')
        limpios, descartados = limpiar_equipos(bruto)
        for _, fila in descartados.iterrows():
            logging.warning('Fila descartada: %s', fila.to_dict())
        limpios = anadir_iva(limpios)
        resumenes = crear_resumenes(limpios, responsables)
        cargar(limpios, descartados, resumenes)
        print('Proceso ETL terminado. Revisa la carpeta resultados/')
    except (FileNotFoundError, KeyError, ValueError) as error:
        logging.error('El proceso ha fallado: %s', error)
        print(f'Error: {error}')


if __name__ == '__main__':
    main()
```

Por qué `logging` y no `print`: el módulo estándar añade fecha, nivel (`INFO`, `WARNING`, `ERROR`) y escribe en archivo. Es lo que se usa en producción, donde nadie está mirando la consola.

11.4 Pruebas con `pytest`

```
python -m pip install pytest
python -m pip freeze > requirements.txt
python -m pytest
```

```
# tests/test_limpieza.py
import pandas as pd
from main import limpiar_equipos


def crear_df(**cambios):
    base = {'codigo': ['A1'], 'dispositivo': ['Portátil'], 'marca': [' hp '],
            'modelo': ['ProBook'], 'ram': [16], 'precio': [800],
            'departamento': ['ventas'], 'estado': ['en uso']}
    base.update(cambios)
```

```
    return pd.DataFrame(base)


def test_normaliza_marca_y_departamento():
    limpios, _ = limpiar_equipos(crear_df())
    assert limpios.loc[0, 'marca'] == 'HP'
    assert limpios.loc[0, 'departamento'] == 'Ventas'


def test_elimina_duplicados():
    df = pd.concat([crear_df(), crear_df()], ignore_index=True)
    limpios, _ = limpiar_equipos(df)
    assert len(limpios) == 1


def test_descarta_precio_no_numerico():
    limpios, descartados = limpiar_equipos(crear_df(precio=['abc']))
    assert len(limpios) == 0
    assert len(descartados) == 1
```

Copilot (`/tests`) puede proponer más casos; **tú decides** cuáles tienen sentido y los ejecutas.

11.5 Flujo de trabajo en Git (nivel empresa)

Momento	Commit sugerido
Estructura	`Crea estructura del proyecto y .gitignore`
Extract	`Implementa extracción y comprobación de rutas`
Transform	`Añade limpieza de datos y registro de descartes`
Resúmenes	`Calcula IVA y resúmenes por departamento y estado`
Load	`Exporta CSV, JSON y Excel`
Pruebas	`Añade tests con pytest`
Docs	`Documenta uso y revisión de Copilot en README`

🔴 **Ampliación:** trabaja con ramas (`git switch -c feature/transform`), abre un *pull request* en GitHub y pide a un compañero que lo revise antes de fusionarlo con `main` .

11.6 Plantilla de `README.md`

```
# Proyecto ETL de inventario informático

## Descripción
Qué hace el programa y qué archivos genera.

## Requisitos e instalación
python -m venv .venv
.venv\Scripts\Activate.ps1
python -m pip install -r requirements.txt

## Ejecución
python main.py
python -m pytest

## Decisiones de limpieza
Qué haces con nulos, duplicados y precios inválidos, y por qué.

## Uso de GitHub Copilot
Tabla de bitácora: qué pedí, qué sugirió, qué comprobé o cambié.

## Autoría
Nombre y grupo.
```

11.7 Niveles de entrega y rúbrica

Nivel	Entrega
🟢 Mínimo	Extract y Load funcionan; limpieza básica; un CSV y un Excel de dos hojas
🟡 Completo	Todo lo del enunciado: JSON, Excel con 3 hojas, `logging`, incidencias, `merge` y README
🔴 Ampliación	Tests con `pytest`, formato Excel con openpyxl, ramas y *pull request*, descarga desde API con `requests`, `uv`

Criterio	Peso
Extract: rutas, lectura y comprobaciones	15 %
Transform: limpieza, IVA, agrupaciones y `merge`	25 %

Criterio	Peso
Load: CSV, JSON, Excel y trazabilidad	20 %
Calidad del código: funciones, *type hints*, nombres, `logging`	15 %
Git y reproducibilidad: commits, `.gitignore`, `requirements.txt`, README	15 %
Explicación individual y uso responsable de Copilot	10 %

🤖 Copilot en el proyecto

Puedes usar el modo **Agent** para tareas acotadas (por ejemplo, «*crea el esqueleto de* `tests/test_limpieza.py`»), siempre revisando el *diff* antes de aceptar. Documenta en el README **al menos tres** interacciones: una en la que te ayudó, una en la que se equivocó y cómo lo detectaste, y una en la que rechazaste su sugerencia.

Anexo A · Evaluación de la unidad

Evidencia	Peso
Prácticas de archivos, rutas, CSV y JSON (Bloques 1-5)	30 %
Prácticas de pandas, Excel y limpieza (Bloques 6-9)	35 %
Proyecto integrador ETL (Bloque 10)	25 %
Explicación individual, pruebas y uso responsable de Copilot	10 %

Explicación individual (oral, 5 min): el docente elige un fragmento de tu entrega y te pide que expliques qué hace, qué ocurre si el dato es incorrecto y cómo lo comprobaste. Si no puedes explicarlo, no cuenta como tuyo, venga de Copilot o de un compañero.

Resultado de aprendizaje esperado: leer, guardar, validar, transformar y analizar datos de archivos comunes (TXT, CSV, JSON, Excel); consumir una API sencilla; explicar el código; y entregar un proyecto reproducible y versionado en GitHub.

Anexo B · Chuleta rápida

Tarea	Código
Leer texto	`with open(ruta, 'r', encoding='utf-8') as f:`
Añadir al final	`open(ruta, 'a', encoding='utf-8')`
Ruta anclada al proyecto	`BASE_DIR = Path(__file__).resolve().parent`
Crear carpeta	`Path('resultados').mkdir(exist_ok=True)`
Buscar CSV	`Path('datos').glob('*.csv')`
CSV como diccionarios	`csv.DictReader(f)` (con `newline=''`)
JSON → Python	`json.load(f)` / `json.loads(texto)`
Python → JSON	`json.dump(obj, f, indent=4, ensure_ascii=False)`
API	`requests.get(url, timeout=5)` + `raise_for_status()`
CSV → DataFrame	`pd.read_csv(ruta)`
Excel → DataFrame	`pd.read_excel(ruta, sheet_name='Hoja1')`
Nulos / duplicados	`df.isna().sum()` / `df.duplicated().sum()`
Texto a número	`pd.to_numeric(col, errors='coerce')`
Filtrar	`df[(df['a'] > 1) & (df['b'] == 'x')]`
Agrupar	`df.groupby('dep').agg(total=('precio', 'sum'))`
Unir tablas	`df.merge(otro, on='clave', how='left')`
Exportar	`df.to_csv(ruta, index=False)` / `df.to_excel(ruta, index=False)`

Anexo C · Script del profesor para generar los datos de práctica

Ejecútalo una vez en la raíz del proyecto para crear `equipos_grande.csv` (80 equipos con departamentos de tamaño muy distinto), `inventario_bruto.csv` (versión sucia) y `responsables.json`.

```
import csv
import json
import random
from pathlib import Path

random.seed(42)
Path('datos').mkdir(exist_ok=True)

DEPARTAMENTOS = ['Administración', 'Sistemas', 'Ventas', 'Dirección',
'Soporte', 'Marketing']
PESOS = [30, 20, 14, 10, 5, 1]
MODELOS = {
    'Lenovo': ['ThinkPad E14', 'ThinkCentre M70', 'ThinkPad T14'],
    'HP': ['ProBook 450', 'ProDesk 400', 'EliteDesk 800'],
    'Dell': ['Latitude 5440', 'OptiPlex 7010', 'Precision 3580'],
    'Asus': ['ExpertBook B1'],
}
ESTADOS = ['Disponible', 'En uso', 'En uso', 'En uso', 'En reparación']
PRECIO_BASE = {8: 500, 16: 700, 32: 950, 64: 1400}
NOMBRES = ['Marta Gil', 'Iván Ríos', 'Laura Sanz', 'Pablo Núñez', 'Elena
Mora', 'Hugo Prieto']


def generar_equipo(i: int) -> dict:
    marca = random.choice(list(MODELOS))
    ram = random.choice(list(PRECIO_BASE))
    return {
        'codigo': f'PC{i:04d}',
        'dispositivo': random.choice(['Sobremesa', 'Portátil']),
        'marca': marca,
        'modelo': random.choice(MODELOS[marca]),
        'ram': ram,
        'precio': PRECIO_BASE[ram] + random.randint(-80, 150),
        'departamento': random.choices(DEPARTAMENTOS, weights=PESOS)[0],
        'estado': random.choice(ESTADOS),
    }


CAMPOS = list(generar_equipo(1))
equipos = [generar_equipo(i) for i in range(1, 81)]


def escribir_csv(ruta: str, filas: list[dict]) -> None:
    with open(ruta, 'w', encoding='utf-8', newline='') as archivo:
        escritor = csv.DictWriter(archivo, fieldnames=CAMPOS)
        escritor.writeheader()
```

```
        escritor.writerows(filas)


escribir_csv('datos/equipos_grande.csv', equipos)

# Versión sucia: duplicados, precios corruptos, nulos, espacios y mayúsculas
sucios = [dict(e) for e in equipos]
sucios += [dict(e) for e in random.sample(sucios, 5)]
sucios[3]['precio'] = 'abc'
sucios[10]['precio'] = ''
sucios[15]['codigo'] = ''
sucios[20]['ram'] = ''
for e in sucios[25:35]:
    e['marca'] = random.choice([' ' + e['marca'].lower() + ' ',
e['marca'].upper()])
for e in sucios[40:45]:
    e['estado'] = e['estado'].lower()
escribir_csv('datos/inventario_bruto.csv', sucios)

responsables = [{'departamento': d, 'responsable': n}
                for d, n in zip(DEPARTAMENTOS, NOMBRES)]
with open('datos/responsables.json', 'w', encoding='utf-8') as archivo:
    json.dump({'responsables': responsables}, archivo, indent=4,
ensure_ascii=False)

print('Datos de práctica generados en datos/')
```

> **Consejo docente:** pide al alumnado que ejecute primero el ETL con el CSV de 10 equipos del Bloque 0, y solo después con `inventario_bruto.csv`. Así comprueban que su código funciona con datos limpios antes de enfrentarse a los sucios.